

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO (formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	PRUEBAS DE SOFTWARE				
TÍTULO DE LA PRÁCTICA:	Desarrollo Orientado a Pruebas (TDD) y Comportamiento (BDD)				
NÚMERO DE PRÁCTICA:	02	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	X	MÁXIMO DE ESTUDIANTES	4	
FECHA INICIO:	20/04/2026	FECHA FIN:	24/04/2026	DURACIÓN:	100 min
RECURSOS A UTILIZAR: • Una computadora personal • Lenguaje de programación Python • Visual Studio Code					
DOCENTE(s): • Prof. Robert Arisaca / Prof. Diego Iquira / Prof. Lino Pinto					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: • El estudiante comprenderá y aplicará el ciclo de vida del Desarrollo Orientado a Pruebas (TDD) • El estudiante elaborará especificaciones ejecutables utilizando el enfoque del Desarrollo Orientado al Comportamiento (BDD).	
TEMAS: • TDD (Test-Driven Development): Ciclo de vida y refactorización • BDD (Behavior-Driven Development)	
COMPETENCIAS	C.m. Construye responsablemente soluciones siguiendo un proceso adecuado llevando a cabo las pruebas ajustadas a los recursos disponibles del cliente

CONTENIDO DE LA GUÍA

• MARCO CONCEPTUAL

1. El Paradigma de "Pruebas Primero" (Test-First)

En el desarrollo de software moderno, las pruebas han dejado de ser una fase final para convertirse en una disciplina continúa integrada en todo el ciclo de vida del software.

El enfoque "Pruebas Primero" invierte el modelo tradicional al exigir que los casos de prueba se diseñen y automaticen antes de escribir el código de producción.

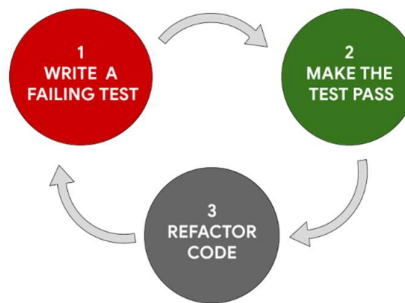
Esta práctica obliga al desarrollador a comprender profundamente la especificación y los requisitos antes de implementar cualquier funcionalidad. Al adoptar este paradigma, los equipos logran un "desplazamiento a la izquierda" (**Shift-Left**), detectando defectos en las etapas más tempranas donde el costo de reparación es significativamente menor.

2. Desarrollo Orientado a Pruebas (TDD)

El TDD es un enfoque de desarrollo de software donde las pruebas unitarias se escriben antes que el código de la aplicación. Esta técnica invierte el proceso tradicional de desarrollo y es un pilar fundamental en metodologías ágiles como la Programación Extrema (XP).

• El Ciclo Red-Green-Refactor:

- **Red (Rojo):** Escribir una prueba pequeña que falle porque la funcionalidad aún no existe.
- **Green (Verde):** Escribir el código mínimo necesario para que la prueba pase.
- **Refactor (Refactorizar):** Mejorar el código (limpieza, optimización) sin cambiar su comportamiento externo, asegurando que las pruebas sigan pasando.



Ejemplo de TDD (Python): Supongamos que queremos una función `es_par(n)`.

1. **Red:** Escribimos la prueba `assert es_par(4) == True`. Al ejecutarla, falla porque la función no existe.
2. **Green:** Definimos la función mínima: `def es_par(n): return True`. Ahora la prueba pasa.
3. **Refactor:** Ajustamos la lógica para que sea correcta para todos los casos: `def es_par(n): return n % 2 == 0`.

El TDD promueve la creación de código modular, desacoplado y con una alta cobertura de pruebas, lo que resulta en aplicaciones más robustas y fáciles de evolucionar.

3. Desarrollo Orientado al Comportamiento (BDD)

El **Behavior-Driven Development (BDD)** es una evolución de TDD que desplaza el enfoque de la implementación técnica hacia el comportamiento del sistema desde la perspectiva del negocio y del usuario. Su objetivo principal es cerrar la brecha de comunicación entre los interesados (stakeholders), evaluadores y desarrolladores.

BDD utiliza especificaciones ejecutables escritas en lenguaje natural estructurado, comúnmente bajo la sintaxis de Gherkin. Estas especificaciones sirven simultáneamente como documentación viva y como pruebas automatizadas. Un escenario estándar de BDD sigue la estructura:

- **Dado (Given):** Describe el contexto inicial o las precondiciones del sistema.
- **Cuando (When):** Define la acción o el evento realizado por el usuario.
- **Entonces (Then):** Establece el resultado esperado o la postcondición que debe verificarse.

Ejemplo de BDD (Gherkin): Escenario: Inicio de sesión exitoso.

- Dado que el usuario está en la página de login.
- Cuando ingresa el usuario "admin" y la contraseña "1234".
- Entonces es redirigido al panel de control (dashboard).

4. Sinergia y Beneficios Estratégicos

Mientras que TDD se centra en la "correctitud" de la lógica interna del código (micro-validación), BDD asegura que el software realmente entregue el valor esperado al cliente (validación de negocio). La integración de ambos enfoques permite construir sistemas que no solo funcionan correctamente desde un punto de vista técnico, sino que también cumplen fielmente con los requisitos del usuario final. En el ecosistema de actual, estas metodologías son fundamentales para sostener tuberías de **Integración Continua (CI)** de alta velocidad, proporcionando retroalimentación instantánea y reduciendo drásticamente el riesgo de regresiones en producción

5. Diferencias Clave entre TDD y BDD

Característica	TDD	BDD
Enfoque	Técnico (¿Cómo funciona el código?)	De negocio (¿Qué hace la funcionalidad?)
Público	Principalmente desarrolladores.	Desarrolladores, testers y analistas de negocio.
Lenguaje	Código de programación (p. ej., Python, Java).	Lenguaje natural estructurado (p. ej., Gherkin).

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

Ejercicio 1 - Implementación de un Validador de Contraseñas usando BDD y TDD

Enunciado: Se requiere desarrollar un módulo de seguridad que valide si una contraseña cumple con las políticas de la empresa:

1. Debe tener al menos 8 caracteres.
2. Debe contener al menos un número.

1. Fase BDD (Definición del Comportamiento)

Antes de programar, definimos el comportamiento esperado utilizando el lenguaje **Gherkin**. Esto permite que el equipo de negocio y desarrollo estén alineados.

Escenario: Contraseña válida

- **Dado** que el usuario ingresa una nueva contraseña.
- **Cuando** la contraseña es "Secure123".
- **Entonces** el sistema debe confirmar que es válida.

Escenario: Contraseña muy corta (Falla)

- **Dado** que el usuario ingresa una nueva contraseña.
- **Cuando** la contraseña es "123".
- **Entonces** el sistema debe informar que es inválida por longitud.

2. Fase TDD (Ciclo de Desarrollo)

Siguiendo el ciclo **Red-Green-Refactor**, aplicaremos pruebas unitarias en Python.

Paso A: Red (Escribir una prueba que falle) Creamos el archivo test_password.py. Como la función validar_password aún no existe, la prueba fallará.

```
import unittest
from auth import validar_password
class TestPassword(unittest.TestCase):
    def test_longitud_minima(self):
        # Esta prueba fallará inicialmente (RED)
        self.assertTrue(validar_password("Segura123"))
        self.assertFalse(validar_password("123"))

unittest.main()
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 5

Paso B: Green (Código mínimo para pasar) Creamos auth.py con la lógica mínima necesaria.

```
def validar_password(password):
    # Código mínimo para que pase la prueba de longitud
    return len(password) >= 8
```

Paso C: Refactor y Nuevos Requisitos Ahora añadimos la validación del número y refactorizamos para que el código sea limpio.

```
def validar_password(password):
    tiene_longitud = len(password) >= 8
    tiene_numero = any(char.isdigit() for char in password)

    return tiene_longitud and tiene_numero
```

3. Validación de Resultados

Ejecutamos las pruebas en la terminal de VS Code: `python test_password.py`

Análisis del Caso de Prueba

Hemos diseñado casos de prueba para:

- **Clases de equivalencia válidas:** "Secure123" (Cumple ambas condiciones).
- **Clases de equivalencia inválidas:** "123" (Falla longitud) y "Password" (Falla número).
- **Valores límite:** Una contraseña de exactamente 8 caracteres.

Ejercicio 2 - Implementación del Módulo de Descuentos mediante TDD

En el subsistema *DreamCar*, la especificación para el cálculo de descuentos en "extras" (equipamiento adicional) dicta lo siguiente:

- Si se eligen **3 o más extras**, se aplica un **10%** de descuento sobre esos artículos.
- Si se eligen **5 o más extras**, el descuento aumenta al **15%**.

Deben desarrollar la función `calcular_descuento_extras(num_extras)` siguiendo estrictamente el flujo **Red-Green-Refactor** de TDD.

Paso 1: Fase RED (Escribir una prueba que falle)

En TDD, las pruebas se diseñan **antes** de programar la lógica. En Visual Studio Code, cree el archivo `test_descuentos.py`:

```
import pytest

from descuentos import calcular_descuento_extras
```

Prueba para la primera regla: 3 extras = 10%

```
def test_descuento_tres_extras():
```

```
    assert calcular_descuento_extras(3) == 10
```

Prueba para la segunda regla: 5 extras = 15%

```
def test_descuento_cinco_extras():
```

```
    assert calcular_descuento_extras(5) == 15
```

- **Acción:** Ejecuta la prueba en la terminal con pytest.
- **Resultado esperado:** La prueba debe fallar con un `ModuleNotFoundError` o `ImportError` porque el archivo `descuentos.py` aún no existe. Esto valida que la prueba es capaz de detectar la ausencia de la funcionalidad [Sources in history].

Paso 2: Fase GREEN (Escribir el código mínimo para pasar)

Cree el archivo `descuentos.py` e implemente la lógica mínima. *Nota técnica: Se introducirá intencionalmente un defecto común mencionado en las fuentes (el orden incorrecto de los IF) para demostrar cómo el TDD lo detecta.*

```
def calcular_descuento_extras(num_extras):
```

```
    if num_extras >= 3:
```

```
        return 10
```

```
    elif num_extras >= 5:
```

```
        return 15
```

```
    return 0
```

- **Acción:** Ejecuta pytest.
- **Resultado:** Verá que la prueba de 3 extras pasa, pero **la de 5 extras falla**. El programa devuelve 10 en lugar de 15 porque la condición `num_extras >= 3` atrapa el valor 5 primero.

Paso 3: Fase REFACTOR (Corregir y optimizar)

Ajuste la lógica para cumplir con todas las especificaciones y aplique robustez (análisis de valores límite de Myers).

Código final en `descuentos.py`:

```
def calcular_descuento_extras(num_extras):
```

```
    # Refactorización: Validación de entrada (Myers - Robustez)
```

```
if not isinstance(num_extras, int) or num_extras < 0:
    raise ValueError("El número de extras debe ser un entero positivo")

# Corrección de la lógica: Las condiciones más restrictivas van primero
if num_extras >= 5:
    return 15
if num_extras >= 3:
    return 10

return 0
```

Paso 4: Validación final

Actualice sus pruebas para incluir **particiones de equivalencia** y **valores límite**:

```
def test_limites_y_particiones():
    # Valores límite (Myers)
    assert calcular_descuento_extras(2) == 0 # Límite inferior de la clase 10%
    assert calcular_descuento_extras(3) == 10 # Frontera exacta
    assert calcular_descuento_extras(4) == 10 # Dentro de la clase
    assert calcular_descuento_extras(5) == 15 # Salto de frontera
```

```
def test_entradas_invalidas():
    # Prueba de robustez (Error Guessing)
    with pytest.raises(ValueError):
        calcular_descuento_extras(-1)
```

Al ejecutar pytest, todas las pruebas deben aparecer en **verde**, confirmando que el código cumple con la especificación y es resistente a errores.

```
python -m pytest test_descuentos.py
```

Asegúrense de tener instalado pytest. Si no lo tienen pueden instalarlo en la terminal con:

```
pip install pytest
```

Unittest y Pytest

1. Unittest

Es el framework que viene integrado por defecto en Python. Se basa en el modelo de JUnit (de Java), lo que lo hace muy familiar para quienes vienen de otros lenguajes orientados a objetos.

Arquitectura: Se basa en clases. Cada "Suite" de pruebas es una clase que hereda de `unittest.TestCase`.

Enfoque de Myers: Es ideal para aplicar el diseño de casos de prueba formal. Al ser tan estructurado, te obliga a separar claramente qué estás probando, facilitando la identificación de particiones de equivalencia.

Lo usamos en el ejercicio de `auth.py` (validador de contraseñas). Es excelente para aprender la disciplina de las pruebas porque su sintaxis es explícita (ej. `self.assertTrue`, `self.assertEqual`).

2. Pytest

Es un framework externo (requiere instalación) que se ha convertido en el favorito de los desarrolladores modernos por su simplicidad y potencia.

Arquitectura: Basada en funciones. No necesitas escribir clases ni aprender métodos especiales de comparación; usas el `assert` estándar de Python.

Enfoque de Spillner: Spillner enfatiza la automatización eficiente. `pytest` brilla aquí porque permite ejecutar pruebas de forma mucho más rápida y su reporte de errores es extremadamente detallado (te dice exactamente qué falló y por qué).

Lo usamos en el ejercicio de `descuentos.py`. Permite implementar el ciclo Red-Green-Refactor de forma ágil, y su manejo de valores límite es muy limpio mediante el uso de "parametrización" (probar muchas entradas con una sola función).

Característica	unittest	pytest
Instalación	Ninguna (incluido en Python).	Requiere <code>pip install pytest</code> .
Sintaxis	Orientada a Objetos (Clases).	Funcional (más simple).
Aseveraciones	Métodos propios (<code>assertEqual</code>).	<code>assert</code> nativo de Python.
Reportes	Básicos (puntos y texto).	Ricos (colores y detalles de variables).

Versatilidad

Rígido pero muy ordenado.

Flexible y altamente extensible.

III. EJERCICIO/PROBLEMA PROPUESTOS

Para cada ejercicio, el grupo deberá entregar el código de la lógica, el código de las pruebas y capturas de pantalla que demuestren el ciclo Red-Green-Refactor o la ejecución de los escenarios Gherkin.

1. TDD: Validador de Identificadores (Basado en Myers)

- Desarrolle un módulo que valide si el nombre de una variable (identificador) es válido según las reglas clásicas de Fortran mencionadas por Myers:
 1. Debe tener entre 1 y 6 caracteres.
 2. El primer carácter debe ser obligatoriamente una letra.
 3. Los caracteres restantes pueden ser letras o dígitos.
- Aplicar TDD para implementar las reglas una por una. Se deben diseñar casos de prueba para los valores límite (longitud 0, 1, 6 y 7) y para la robustez (caracteres especiales).
- Entregables: `identificador.py` y `test_identificador.py`.

2. BDD: Proceso de Retiro en Cajero Automático (Caso ATM)

- Utilizando la lógica del simulador de Cajero Automático (ATM) de la sesión anterior, aplicaremos BDD (Behavior-Driven Development) para validar que el sistema se comporte correctamente ante errores del usuario.
- Deben definir y probar el comportamiento del cajero cuando un usuario intenta retirar más dinero del que tiene disponible. Según Myers, este es un caso de prueba de "error" necesario para garantizar la robustez del software.
- Tareas a realizar:
 - Redactar la especificación (Gherkin) creando un archivo llamado `atm.feature` donde expliques el fallo usando el formato:
 - Dado que mi saldo es 100.
 - Cuando intento retirar 150.
 - Entonces debo ver un mensaje de "Fondos Insuficientes".
 - Implementar la prueba, deben traducir esos pasos a código Python para verificar que el simulador realmente bloquee la transacción.
- Entregables: `atm.feature` y `test_atm.py`.

3. Validador de Calificaciones

- Se requiere una función evaluar_rendimiento(nota) que reciba un número entero entre 0 y 20.
- **Requisitos:**
 - 0 - 10: "Insuficiente"
 - 11 - 15: "Regular"
 - 16 - 20: "Excelente"
- El estudiante debe aplicar TDD para manejar los rangos y lanzar una excepción (ValueError) si la nota está fuera del rango 0-20.
- Identificar clases de equivalencia válidas e inválidas.

4. Calculadora de Reembolsos

Deben crear un sistema que calcule cuánto dinero se le devuelve a un cliente cuando cancela una reserva de hotel, aplicando reglas especiales para clientes VIP.

Reglas de Negocio:

- **Cancelación con mucha antelación (> 72 horas):** Se devuelve el 100%.
- **Cancelación con antelación media (24 a 72 horas):** Se devuelve el 50%.
- **Cancelación de última hora (< 24 horas):** Se devuelve el 0%.
- **Regla VIP:** Si el cliente es VIP, siempre se le devuelve al menos el 50%, sin importar cuándo cancele.

Tareas del Estudiante:

1. **Diseño BDD (Gherkin):** Escriban un escenario donde un cliente VIP cancela con solo 2 horas de antelación y recibe su 50% de reembolso.
2. **Pruebas de Robustez (TDD):** Creen pruebas que verifiquen:
 - Los valores límite (¿qué pasa exactamente a las 24 y 72 horas?).
 - Protección contra errores: El sistema debe lanzar un error si el monto de la reserva es negativo.
3. **Implementación:** Programa la función calcular_reembolso(monto, horas, es_vip) asegurándose de que la regla VIP tenga prioridad sobre la regla de última hora.

Entregables:

- Archivo de texto con el escenario Gherkin y Código de las funciones y sus pruebas ejecutadas con éxito.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 11

I. CUESTIONARIO

1. ¿Por qué en TDD se considera que "la prueba es el primer cliente del código"?
2. Explique cómo el enfoque BDD ayuda a evitar errores de comunicación entre el Product Owner y el equipo de desarrollo.
3. Según Myers, ¿qué es el "análisis de valor límite" y por qué es más efectivo que probar solo valores aleatorios?
4. ¿Cuál es el propósito del paso Refactor en el ciclo de TDD y qué riesgo se corre si se omite?
5. Explique la importancia de definir el resultado esperado antes de la ejecución de una prueba.

II. REFERENCIAS Y BIBLIOGRAFÍA RECOMENDADAS:

- [1] Andreas Spillner, "Software Testing Foundations", Editorial Rocky Nook, 2021, 5th Edition
[2] Glenford J. Myers, "The Art of Software Testing", Editorial Jhon Wiley & Sons, 2012, 3th Edition

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:

*Ejercicios propuestos /
Cuestionario*

INSTRUMENTOS:

Rúbrica

CRITERIOS DE EVALUACIÓN

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (55%)	Insuficiente (30%)	No presenta (0%)
Claridad y completitud de la explicación de los ejercicios propuestos. (30 %)	Todos los ejercicios propuestos han sido explicados de forma muy clara y con la profundidad adecuada.	Los ejercicios propuestos han sido resueltos y explicados, pero de manera muy superficial o general.	Los ejercicios propuestos han sido resueltos, sin embargo, algunos de ellos no han sido explicados.	Los ejercicios han sido resueltos parcialmente y no tienen explicaciones de lo realizado.	No presentan los ejercicios resueltos, o la versión final del informe omite por completo esta sección.
Ejercicios Propuestos (Código fuente) (20 %)	Todos los ejercicios propuestos se ejecutan correctamente y devuelven los resultados esperados.	Los ejercicios propuestos han sido implementados pero con los resultados esperados parciales.	Por lo menos el 2/3 de los ejercicios propuestos han sido implementados con resultados	Por lo menos el 1/3 de los ejercicios propuestos han sido implementados con resultados	No se han implementado los ejercicios propuestos.
Cuestionario (30%)	Todas las respuestas son completas,	La mayoría de las respuestas son correctas,	Algunas respuestas son correctas, pero	La mayoría de las respuestas son escuetas	No presenta el cuestionario.

	correctas y proporcionan explicaciones claras, incluyen citas.	pero algunas carecen de la profundidad esperada, pocas citas.	incompletas o con falta de claridad en las explicaciones, citas no relevantes.	y/o incompletas, no hay evidencia de haber buscado información que las respalden, sin citas.	
Presentación del Informe (20%)	El informe está bien estructurado, con excelente redacción, sin errores	El informe está bien estructurado, con algunos errores menores de redacción	El informe presenta problemas en la organización o redacción, con varios errores	El informe está desorganizado, con múltiples errores de redacción y formato	No presenta el informe.